The Relay: Resilient Research Paper Assistant

Author: Udbhaw Anand | **Repository:** https://github.com/Udbhaw08/the-relay

Deliverable File: `data/runs/phase9_final_verified/results.csv` (125 evaluated requests, Seed 42, Run ID `phase9_final_verified`) | **Historical Fixture:** `results.csv`

1. What I built and why

I built **The Relay**, an agentic research assistant in Python answering technical engineering queries over three arXiv papers (*Attention Is All You Need*, *Moshi*, and *Dynamic Chunking for H-Net*). The system executes four decoupled stages: (1) Query parsing & out-of-scope intent detection, (2) Hybrid passage retrieval combining BM25Okapi lexical matching with dense vector search merged via Reciprocal Rank Fusion (RRF, $k=60$), (3) Synthesis and citation drafting requiring exact bracketed page/section anchors `[Paper, Page X, Section Y]`, and (4) an automated Citation Verification and Factual Guard that validates groundedness, triggers single query-expansion loop-backs on missing citations, and strictly refuses unanswerable out-of-scope questions.

The One Decision That Shaped It Most:

Enforcing **zero silent fallback with adaptive retrieval degradation**. Upstream provider limits are never masked by silently routing to unannounced third-party models. Instead, when rate limits or queue pressures occur, the pipeline throttles retrieval from Setting B/C to compact Setting A (reducing prompt context tokens by ~70%), keeping the service available while preserving strict telemetry provenance.

2. The real service and my stand-in

The system targets **GroqCloud API** (`api.groq.com/openai/v1`) using model `openai/gpt-oss-20b` (following the decommissioning of legacy `llama-3.3-70b-versatile` on August 16, 2026). All benchmark evaluations were executed using our calibrated **Stand-in Simulator** because the friend's account authorization and Free-tier status remain unverified.

Table 1: Published Groq Free-plan limits (documentation; not observed on the key owner's account)

Limit	Value you found	How you found it	Confidence (high, medium, low)
Requests per minute	30 RPM (published Free plan)	Official Groq Documentation (openai/gpt-oss-20b published tier)	high
Tokens per minute	8,000 TPM (published Free plan)	Official Groq Documentation & x-ratelimit-limit-tokens header	high
Requests per day	1,000 RPD (published Free plan)	Official Groq Documentation & x-ratelimit-limit-requests header (Groq documents this header as RPD)	high
Tokens per day	200,000 TPD (published Free plan)	Official Groq Documentation (openai/gpt-oss-20b published tier)	high
Requests in flight at once	Dynamic / 1-5 concurrent (unverified friend account)	Groq Cloud Documentation / unverified organization Limits page	medium
What it returns when a limit is hit	HTTP 429 rate_limit_exceeded	Groq API error payload specification & error classifier	high
Time until it recovers	Dynamic via 'retry-after' / 'x-ratelimit-reset-*' headers	x-ratelimit-reset-tokens (minute) / x-ratelimit-reset-requests (day) / retry-after	high

Header Semantics & Caps: Groq documents `x-ratelimit-limit-requests` as Requests Per Day (RPD), NOT RPM. `x-ratelimit-limit-tokens` corresponds to TPM. Local safety caps default conservatively (30 RPM, 8K TPM, 1K RPD, 200K TPD) to protect shared organization quotas. The friend's organization Limits page remains the source of truth for account allocations.

Stand-in Methodology: Latency is generated via a truncated Gaussian distribution ($\mu=1200\text{ms}, \sigma=300\text{ms}, \text{min}=150\text{ms}$) with a token generation rate of 15ms/token (~66 tok/s). Contention is modeled via $\text{TTFT}_{\text{eff}} = \text{TTFT} \times (1 + 0.08 N_{\text{in-flight}}^{1.15})$. TPM and TPD limits are enforced via real tracked token consumption rather than manual booleans. As live Groq access was unverified, this is a synthetic model rather than an empirically calibrated trace; it cannot simulate Groq's hardware LPU SRAM compilation stalls or prompt prefix caching.

3. Results

All 125 benchmark requests were executed offline under run ID `phase9_final_verified` (seed 42), logging to `data/runs/phase9_final_verified/results.csv`.

Table 2a: Latency, one row per load scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token (Simulated TTFT) p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady	stand-in	10	70.9s (17.88s dispatch)	30.2 (interval 1.99s)	1840.4 / 2132.8 / 2137.5 ms	13519.8 / 16109.8 / 16239.5 ms
burst	stand-in	20	66.2s (59.57s dispatch)	Burst (20 concurrent)	2630.0 / 3760.0 / 3983.5 ms	6030.1 / 58282.0 / 58680.8 ms
sustained_overload	stand-in	25	120.1s (115.64s dispatch)	13.0	1515.8 / 2334.6 / 2389.2 ms	5300.9 / 54717.5 / 54978.1 ms
per_minute_limit_halved	stand-in	15	236.7s (194.12s dispatch)	4.6	1393.0 / 1666.8 / 1700.4 ms	3958.4 / 23743.8 / 38788.2 ms
daily_budget_exhausted	stand-in	10	2.0s	306.1	N/A	N/A

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
steady	70.0%	30.0%	0.0%	0.3	5	41.1	N/A — no live provider usage
burst	65.0%	25.0%	10.0%	0.95	20	36.1	N/A — no live provider usage
sustained_overload	40.0%	36.0%	24.0%	0.96	15	33.1	N/A — no live provider usage
per_minute_limit_halved	73.3%	20.0%	6.7%	0.6	0	37.0	N/A — no live provider usage
daily_budget_exhausted	0.0%	100.0%	0.0%	0.0	0	0.0	N/A — no live provider usage

Metric Provenance: Token counts reflect estimated tokens (word/char heuristic). Actual provider token fields are null because no live API calls were made. Cost is labeled "N/A — no live provider usage". First token latency is labeled as Simulated TTFT and is measured exclusively on streaming attempts (non-streaming requests set `first_token_ms = null`).

Table 3: Retrieval settings (at least three)

Setting (piece size, pieces fetched, re-rank yes or no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token (Simulated TTFT) p50 / p95 (ms)	Total time p50 / p95 (ms)
Piece size 250, 3 fetched, re-rank: no	15	8.3% (1/12)	91.7% (11/12)	100.0% (3/3)	39.1	1418.0 / 1558.0 ms	3687.8 / 4835.5 ms

Setting (piece size, pieces fetched, re-rank yes or no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token (Simulated TTFT) p50 / p95 (ms)	Total time p50 / p95 (ms)
Piece size 500, 5 fetched, re-rank: yes	15	8.3% (1/12)	91.7% (11/12)	100.0% (2/2)	36.9	1257.8 / 1539.1 ms	3773.9 / 5125.3 ms
Piece size 1000, 7 fetched, re-rank: yes	15	8.3% (1/12)	91.7% (11/12)	100.0% (3/3)	33.1	1418.0 / 1573.3 ms	3617.9 / 4767.6 ms

Detailed Retrieval Accuracy & Failure Breakdown

Setting	Ans Evaluated	Correct & Cited	Partial	Factually Wrong	Wrongly Refused	Unans Evaluated	Correctly Refused	Hallucinated	Infra Failures
setting_a	12/12	1	0	11	0	3/3	3	0	0/15
setting_b	12/12	1	0	10	1	3/3	3	0	0/15
setting_c	12/12	1	0	8	3	3/3	3	0	0/15

Denominator & Isolation Notice: Pure retrieval benchmarks run with adaptive degradation disabled (`allow_degradation=False`) so Setting A cannot contaminate Settings B or C (15/15 pure requests per setting). Infrastructure failures are excluded from factual accuracy denominators and reported separately. Answerable accuracy (12 questions) and unanswerable refusal accuracy (3 questions) are segregated with exact sample counts.

Chosen Default Setting: Setting B (500 tokens, 5 fetched, hybrid RRF re-ranking) is the default. It balances semantic context depth against prompt token cost. Under rate-limit warnings, the engine dynamically degrades to **Setting A** (250 tokens, 3 fetched, BM25-only), reducing context tokens by ~70%.

An Unexpected Number: In steady load, bounded concurrent scheduling achieved exact 1.0s target start-to-start pacing (**mean measured interval: 1.0s**, **mean dispatch lateness: 6.66ms**) despite cold-start retrieval latencies reaching 13.5s. In Scenario 4, halving baseline RPM to 7 actively exercised rate limiting (20.0% rate-limited, 73.3% degraded success), confirming throttling is operational.

4. Where the time goes

Profiling slow request `steady_stand-in_001` (Total latency: **15,731.4 ms**):

- **Queue wait time:** 0.3 ms (admitted immediately).
- **Retrieval time:** 12,831.8 ms (**81.6% of duration**). Dense embedding cold-start across all corpus chunk partitions.
- **Inference time:** 2,899.2 ms (18.4% of duration: simulated TTFT 2,138.7 ms, generation 760.5 ms across 46 tokens).
- **Verification time:** 0.3 ms (<0.01%: regex extraction and page/anchor verification).
- **Warm comparison:** Once vector weights are memory-resident (`steady_stand-in_009`), retrieval drops to **10.6 ms** (99.9% reduction), reducing total latency to 2,047.9 ms.

5. When a limit is hit

- **Per-Minute Limit (RPM/TPM):** Handled via an in-memory queue. Requests execute truncated exponential backoff with full jitter ($T \sim U(0, \min(M, B \times 2^k))$) and adaptively throttle retrieval to Setting A (250 tokens, BM25-only), shedding ~70% of prompt token load.
- **Per-Day Limit (RPD/TPD):** Stops retries immediately (0 retries). Requests fail fast (<1ms) with an explicit notice stating dynamic reset estimation, preventing thundering-herd queue lockup.

6. How this library will mislead a naive retriever

1. **Lexical Collision on "Attention":** Queries regarding *Moshi's* temporal stream masking match Vaswani's translation masking in *Attention Is All You Need*, returning translation equations instead of audio synchronization.
1. **Term Collision on "Chunking":** Queries on dynamic chunking benefits over tokenizers match *Moshi's* 80ms acoustic frame stride rather than *H-Net's* byte-level hierarchy.
1. **Vocabulary Mismatch on "Tokens":** Asking about discrete audio speech tokens retrieves BPE subwords from Vaswani et al. rather than Mimi neural codec RVQ codebooks.

7. Retries

- **Policy:** Truncated exponential backoff with true full jitter ($T \sim U(0, \min(30.0, 2.0 \times 2^k))$), maximum 2 retries, respecting provider `Retry-After` headers up to a 60s safety cap.
- **Where retrying made things worse:** Retrying on **daily budget exhaustion** caused catastrophic queue saturation. Classifying daily exhaustion as a non-retryable error eliminated retry loops completely.

8. What the person sees

When daily quota is depleted, the user receives an explicit, transparent notice:

> "⚠️ *Daily Query Budget Exhausted: The system's daily LLM inference capacity has reached its limit for today. The next quota allocation resets dynamically (or when upstream headers indicate). Your request was rejected locally to protect shared quotas. Please consult the local paper index or retry once the quota window refreshes.*"

Note on Extended Analysis: Comprehensive phase execution matrices, architectural diagrams, extended reflections, feedback on Vaani, and assistant collaboration logs are documented in `docs/implementation/FINAL_REPORT.md`.